

How data moves through the system

A visual workflow of every component, and the triggers that make things happen on their own.

OpsDesk watches the data-center hardware, turns problems into tickets, gets each ticket to the right engineer, keeps the SLA clock, tells people what changed, and records everything. This document follows the data step by step, in plain language.

- 1 The whole system on one page
- 2 Flow A - Hardware health: from a server to the CMDB page
- 3 Flow B - Alerts: from a monitoring tool to a ticket
- 4 Flow C - The life of an incident
- 5 Flow D - Getting the ticket to an engineer, and the SLA clock
- 6 Flow E - Notifications, audit trail and dashboards
- 7 Every automatic trigger in one table
- 8 One full story: a disk starts to fail

Colour key used on every page

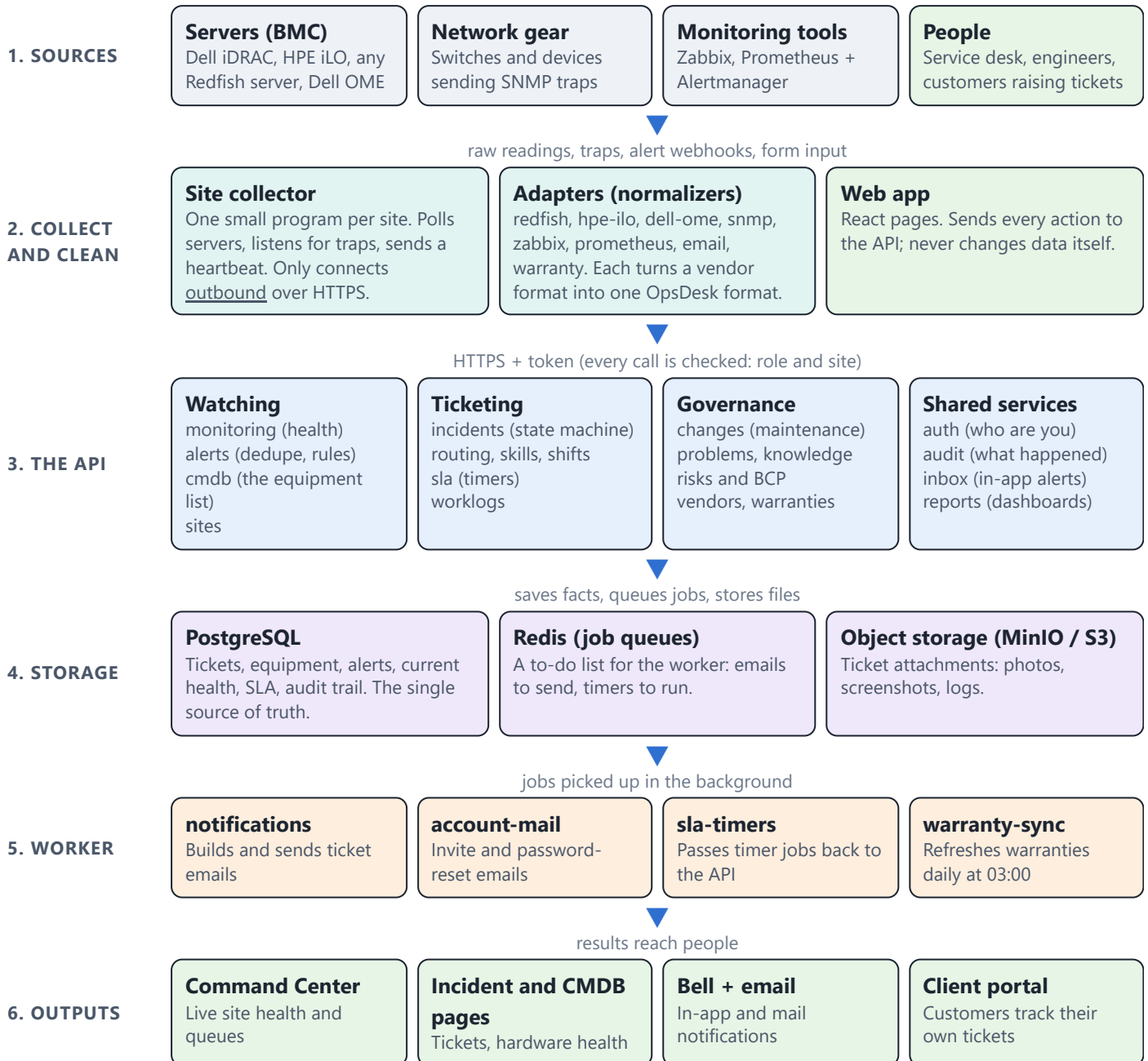


Based on the code in the repository as of 5 October 2026 (branch ankit_workspace).

SECTION 1

The whole system on one page

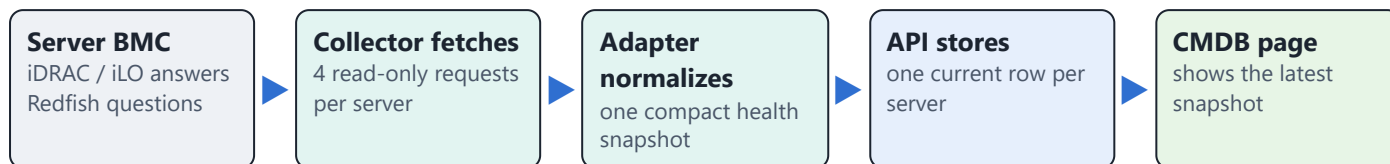
Read it top to bottom. Data starts at the hardware and monitoring tools, is cleaned up, reaches the API, is stored, and finally shows up for people.



Three rules that explain most of the design. (1) Only the API changes data, and it checks permissions on every call. (2) Every important change writes an audit record that can never be edited. (3) Raw sensor history is not stored here; only the current health and the alerts are. Graphs and logs stay in Prometheus, Grafana and Loki.

Hardware health: from a server to the CMDB page

This is what fills the "Hardware health" panel (drives, fans, power supplies) on a server's page.



- 1 **TRIGGER: timer, every pollIntervalSeconds (default 300 s)**
The collector wakes up and goes through its list of servers for that site.
`apps/collector - runHealthPoll()`
- 2 **It asks each server four things** (read only, never changes the server): the system itself, fans and temperatures, power supplies, and every drive (up to 64).
`/redfish/v1/Systems, /Chassis/.../Thermal, /Chassis/.../Power, Storage → Drives`
- 3 **The adapter works out the health.** See the calculation box below.
`integrations/redfish - normalizeRedfishSystem()`
- 4 **The snapshot goes into a local buffer**, then is posted to the API. If the API is unreachable the buffer keeps it (on disk if configured) and retries, oldest first.
`POST /api/v1/monitoring/health-snapshots`
- 5 **The API saves it** as the server's current health (replacing the previous one) and writes an audit record. A server the CMDB does not know is rejected.
`monitoring module - recordSnapshots()`
- 6 **TRIGGER: health is WARNING or CRITICAL**
An alert is opened (type `hardware.health_degraded`) and enters Flow B. Later bad polls update the same alert.
TRIGGER: health is HEALTHY again the same alert is marked recovered.

How the health is calculated

The server says	OpsDesk shows	Special rule	Effect
OK	HEALTHY	Part reported "Absent"	Ignored (empty slot)
Warning	WARNING	Temperature at or above its critical limit	Forced to CRITICAL
Critical	CRITICAL	Drive says "failure predicted"	At least WARNING
anything else / no answer	UNKNOWN		

Overall health = the worst single thing found. One critical fan makes the whole server CRITICAL. The page shows counts such as "Drives 4/4 healthy" and lists only the parts that are not healthy.

"Real time" here means "as of the last poll". Nothing streams. The page shows the newest snapshot when it loads; the "Last reported" time tells you how fresh it is. The collector also sends a **heartbeat** (default every 60 s) so a silent site is not mistaken for a healthy one.

Adding another site

Create the site and its servers in OpsDesk → issue an API token → run one collector inside that site's network with its own list of servers → give it the BMC logins as environment variables. Nothing at the site is exposed to the Internet.

Alerts: from a monitoring tool to a ticket

Many tools can raise an alert. They all end up in one place and go through the same nine checks.

Four doors in, one pipeline



each adapter converts its format into one "normalized alert" → `AlertsService.ingest()`

- 1 **Normalize.** Same fields whatever the source: site, equipment (CI), alert type, severity, state, time.
- 2 **Remove duplicates.** The same event sent twice is recognised by its event ID and by a fingerprint (site + equipment + alert type + component). It updates the existing alert instead of creating a new one.
- 3 **Find the site and the equipment** in the CMDB.
- 4 **Look up the alert rule.** Rules live in the database, not in code: which severities page the NOC, which create a ticket, what happens during maintenance.
- 5 **CHECK: is this equipment in an approved maintenance window?**
If yes and the rule says so, no ticket is opened (the alert is still recorded).
- 6 **TRIGGER: severity is a "paging" severity (default CRITICAL)**
The NOC is notified in the app and by email.
- 7 **Correlate.** If there is already an open ticket for this equipment, the alert is attached to it. No second ticket.
- 8 **TRIGGER: severity is an "auto-create" severity (default CRITICAL) and no open ticket exists**
A new incident is created automatically and enters Flow C.
- 9 **TRIGGER: the alert comes back as RECOVERED**
The alert is closed, and the people on the linked ticket are told the fault has cleared. The ticket itself is not closed automatically; a person confirms.

Why the duplicate check matters. Monitoring tools resend a firing alert every few minutes. Without step 2 you would get a new alert, a new email and a new ticket each time.

Defaults in plain words. A **warning** is recorded and shown. A **critical** alert also emails the NOC and opens a P1 ticket. Both lists can be changed per rule in Administration → Alert rules.

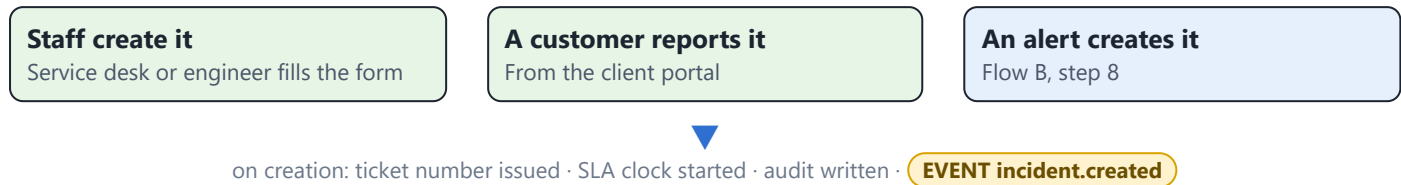
What is stored, and what is not

Stored in OpsDesk	Left in the monitoring tools
The normalized alert, its state, first and last seen time, the linked ticket	CPU graphs, temperature history, raw log lines (Prometheus, Grafana, Loki, Zabbix)

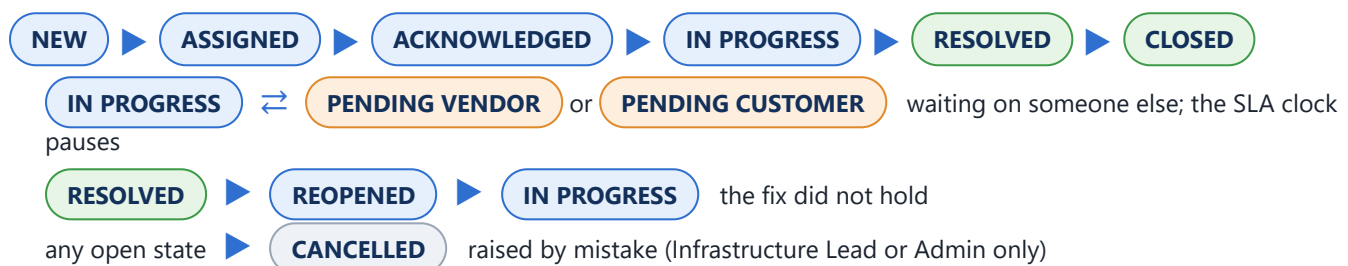
The life of an incident

A ticket can only move along the arrows below. The web app asks; the API decides.

Three ways a ticket is born

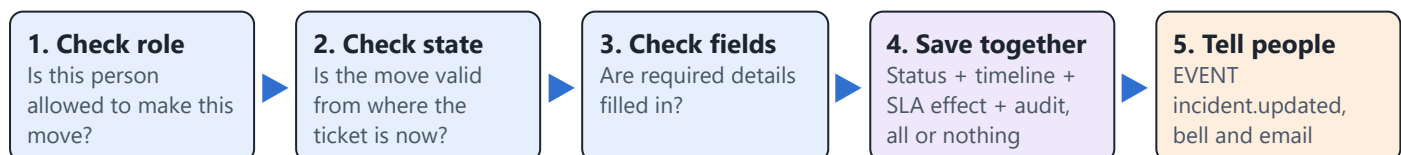


The path



Move	Who can do it	What happens automatically
NEW → ASSIGNED	Service Desk, Infrastructure Lead, Admin (or routing, Flow D)	Owner is notified in app and by email
ASSIGNED → ACKNOWLEDGED	Site Engineer, Infrastructure Lead, Admin	SLA "acknowledge" timer stops
ACKNOWLEDGED → IN PROGRESS	Site Engineer, Infrastructure Lead, Admin	-
IN PROGRESS → PENDING ...	Engineer roles (a reason is required)	SLA clock pauses
PENDING ... → IN PROGRESS	Staff working the ticket	SLA clock resumes
IN PROGRESS → RESOLVED	Site Engineer, Infrastructure Lead, Admin	SLA "resolve" timer stops; customer is told
RESOLVED → CLOSED	Service Desk, Infrastructure Lead, Admin	Ticket is final
RESOLVED → REOPENED	Staff, when the fix did not hold	SLA clock restarts

What the API does on every single move



Who sees which tickets. Admin and Service Desk see every ticket at their sites. A **Site Engineer** sees only tickets they own or are being offered. A **customer** sees only tickets they reported, and never internal notes. This is enforced by the API, not by hiding buttons.

Around the ticket: **comments** (internal or customer-visible), **worklogs** (time an engineer spent; corrections never overwrite the original), **attachments** (stored in object storage), and **vendor cases** for hardware replacement.

Getting the ticket to an engineer, and the SLA clock

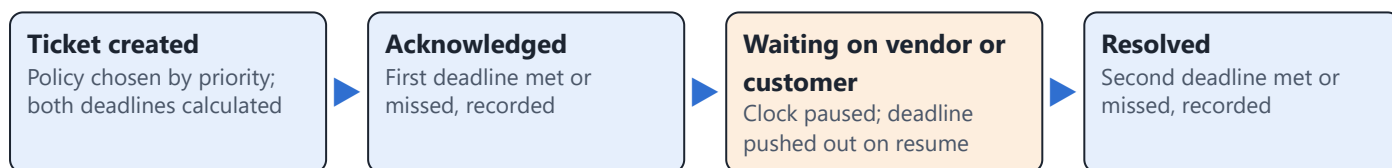
Automatic routing (when switched on for a site)

Instead of the service desk picking someone, OpsDesk offers the ticket to the best available engineer.

- 1 **TRIGGER: event incident.created**
Routing ranks the engineers using three things: the right **skill** for the ticket's category, being **on shift** now, and the lightest **workload** (a P1 counts more than a P4).
- 2 **The top engineer gets an offer** with a time limit that depends on priority (defaults: P1 2 min, P2 5 min, P3 10 min, P4 15 min). They are notified in the app.
- 3 **Accept** → the ticket becomes ASSIGNED to them. **Decline** → it is offered to the next engineer straight away.
- 4 **TRIGGER: timer, every minute**
Expired offers are closed and the ticket moves to the next engineer in the ranking.
- 5 **TRIGGER: nobody is left to offer to**
The service desk is told so a person can assign it by hand.
- 6 **TRIGGER: event incident.updated**
 If someone assigns the ticket manually in the meantime, **the open offer is cancelled**.

The SLA clock

Every ticket carries two deadlines: acknowledge by, and resolve by. Both come from an SLA policy stored in the database for that priority.



Event	Effect on the clock
Priority is changed	Deadlines are recalculated from the new priority's policy
Site has a business-hours calendar	Time outside working hours does not count (for policies that use the calendar)
Ticket is reopened	The resolve clock starts again

Escalation: the warning before the breach

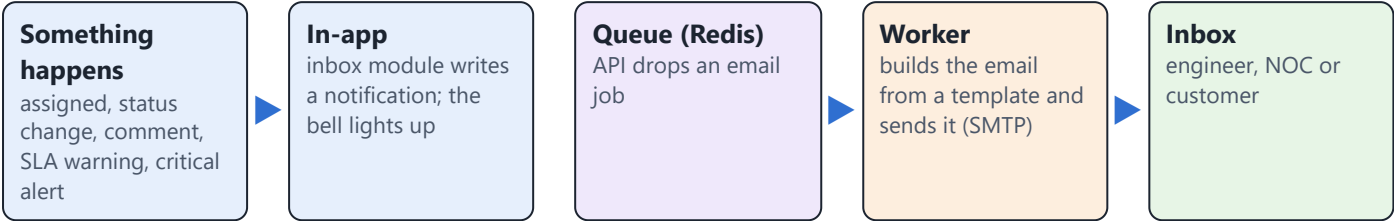
- 1 **TRIGGER: timer, every minute**
The SLA scanner looks at every open ticket and asks: has this ticket just crossed a warning threshold (a set percentage of its allowed time) or its deadline?
- 2 **Each threshold fires once.** It is written on the ticket's timeline and in the audit trail, and the right people are notified in the app and by email.
- 3 **At 100% the ticket is marked breached.** Tickets that have fired a warning but are not yet breached show as "SLA at risk" on the Command Center.

Nothing about time is hard-coded. Targets, warning thresholds, offer time limits, workload weights and support calendars are all settings in the database that an administrator can change.

Notifications, audit trail and dashboards

How a notification reaches a person

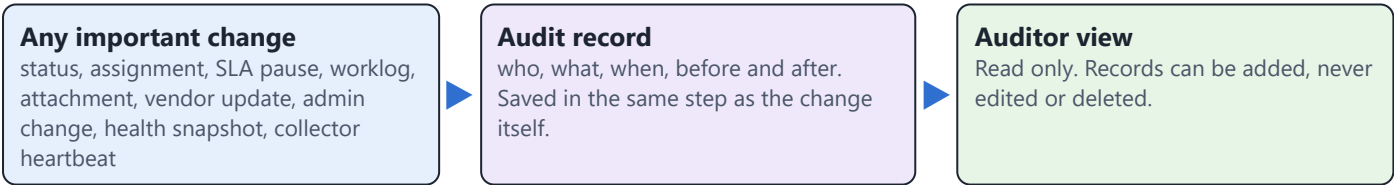
The API never sends email itself. It leaves a job in a queue and carries on, so a slow mail server can never slow down a ticket.



What happened	Who is told
Ticket assigned or offered	The engineer
Status changed	The owner, and the customer who reported it (customer-facing changes only)
Customer-visible comment	The reporting customer; a customer's reply goes to the owner, the group, or the service desk
Internal note	Nobody outside staff; customers never see it
Critical alert	The NOC roster
SLA warning or breach	Owner and escalation contacts
Invite or password reset	That user (account-mail queue)

TRIGGER: daily at 03:00 old in-app notifications are cleaned up. **TRIGGER: daily at 03:00** the worker refreshes hardware warranties from the vendors.

The audit trail



Dashboards and reports

Reports never change anything. Each time a page opens, the reports module reads the current facts and adds them up.

Command Center shows	Calculated from	Other read-outs	What it is
Site health (healthy / warning / critical)	Worst health of any equipment at the site	Insights: response trend	Average time to acknowledge and to restore, per day
Servers reachable	Servers not critical and not unknown	Insights: alert volume	Alerts per day by severity
Critical alerts open	Alerts table	Operational health CSV	The Command Center numbers as a download
P1/P2 open, SLA at risk	Open tickets and their SLA state	Client portal	A customer's own tickets and their site's status
Queues: unassigned, awaiting ack, vendor waiting, reopened	Open tickets by status		

Everyone sees numbers only for the sites they are allowed to see. A Site Engineer's ticket counts cover only their own tickets.

SECTION 7

Every automatic trigger in one table

The database itself has no triggers. All automation is one of four kinds: a **timer**, an internal **event**, a **queue job**, or a **condition** checked while data comes in.

Timers (run on a schedule)

When	Where it runs	What it does
Every poll interval (default 300 s)	Site collector	Polls every server's health and queues the snapshots
Every poll interval	Site collector	Sends anything waiting in the buffer to the API
Every heartbeat interval (default 60 s)	Site collector	Tells the API "this site's collector is alive"
Every minute	API - routing	Expires overdue offers and offers the ticket to the next engineer
Every minute	API - SLA scanner	Finds tickets that crossed a warning threshold or deadline; records and notifies
Daily 03:00	API - inbox	Deletes expired in-app notifications
Daily 03:00	Worker - warranty-sync	Asks the API to refresh warranties from vendor sources

Events (one part of the API tells another)

Event	Raised by	Who reacts, and how
<code>incident.created</code>	incidents, after saving	Routing makes the first offer (if auto-assign is on for the site)
<code>incident.updated</code>	incidents, after saving	Routing cancels an open offer once the ticket is assigned or has moved on
"May this user be deactivated?"	auth (user admin)	Incidents answers no while the user still owns open tickets
"May this user's role change?"	auth (user admin)	Skills and shifts answer no while an engineer still holds skills or active shifts

Queue jobs (API hands work to the worker through Redis)

Queue	Filled when	The worker then
notifications	A ticket or alert event needs an email	Renders the template and sends the mail
account-mail	A user is invited or resets a password	Sends the sign-in link
sla-timers	An SLA timer is scheduled	Relays the timer back to the API when due
warranty-sync	The daily schedule fires	Calls the API's warranty refresh

Conditions (checked the moment data arrives)

If this is true ...	... then this happens
A health snapshot is WARNING or CRITICAL	A hardware alert is opened, or the open one is updated
A health snapshot is HEALTHY and an alert was open	That alert is marked recovered
An incoming alert matches an existing one	The existing alert is updated; no duplicate
The equipment is in a maintenance window	No ticket is opened (if the rule says so)
Alert severity is in the rule's paging list	The NOC is notified
An open ticket already exists for the equipment	The alert is linked to that ticket
Alert severity is in the rule's auto-create list, and no open ticket	A new incident is created
Ticket moves to a PENDING state / leaves it	SLA clock pauses / resumes
The API cannot be reached from a site	The collector keeps the data and retries in order

SECTION 8

One full story: a disk starts to fail

Every flow in this document, in the order it would really happen. Times are an example using the demo P1 policy (acknowledge in 15 minutes, resolve in 4 hours).

- 1 **10:00 A drive in SITE01-R01-SRV-001 reports Critical.** Nobody has noticed yet.
- 2 **10:01 timer The site collector polls the server,** reads the drive status, and the Redfish adapter works out: drives 3/4 healthy, overall CRITICAL.
Flow A, steps 1-3
- 3 **10:01 The snapshot is sent to the API and saved.** The CMDB page for the server now shows CRITICAL with the bad drive listed. An audit record is written.
Flow A, steps 4-5
- 4 **10:01 condition Because the health is CRITICAL, a hardware alert is opened.** It is new, so it passes the duplicate check; the server is not under maintenance.
Flow B, steps 1-5
- 5 **10:01 condition Critical is a paging severity:** the NOC gets a bell notification and an email job goes into the queue. The worker sends the email seconds later.
Flow B step 6, Flow E
- 6 **10:01 condition No open ticket exists for this server, so incident INC-000123 (P1) is created.** The SLA clock starts: acknowledge by 10:16, resolve by 14:01.
Flow B step 8, Flow C
- 7 **10:01 event incident.created Routing ranks the engineers** and offers the ticket to the on-shift engineer with the storage skill and the lightest workload. She has 2 minutes.
Flow D
- 8 **10:02 She accepts.** The ticket becomes ASSIGNED to her and now appears in her incident list. She acknowledges it; the acknowledge deadline is met.
- 9 **10:06 timer The next poll still says CRITICAL.** The same alert is updated. No second alert, no second ticket, no second email.
Duplicate check and correlation
- 10 **10:20 She starts work, logs her time, attaches a photo,** and opens a vendor case for a replacement drive. Ticket moves to PENDING VENDOR. condition The SLA clock pauses.
- 11 **13:00 The drive arrives and is replaced.** Ticket back to IN PROGRESS; the clock resumes with the paused time added back to the deadline.
- 12 **13:11 timer The collector's poll reports HEALTHY.** condition The alert is marked recovered and she is told the fault has cleared.
- 13 **13:15 She marks the ticket RESOLVED** with the cause and fix. The resolve deadline is met. The service desk reviews and CLOSES it.
- 14 **after Everything is on record.** The Command Center shows the site healthy again, Insights includes this ticket in the response-time trend, and the audit trail holds every step above with who did it and when.

What no person had to do: notice the fault, open the ticket, find the right engineer, start or pause the clock, or send a single email. People did the parts that need judgement: accepting the work, fixing the server, and confirming it was fixed.

If the engineer had not accepted in 2 minutes, the minute timer would have expired the offer and passed it to the next engineer. If the ticket had sat unacknowledged, the SLA scanner would have sent warnings before 10:16 and marked a breach at 10:16.